Welcome to CS681

Tue Thu 5:30pm to 6:45pm

Pre-Requisite for CS681

- You need a B or better for CS680.

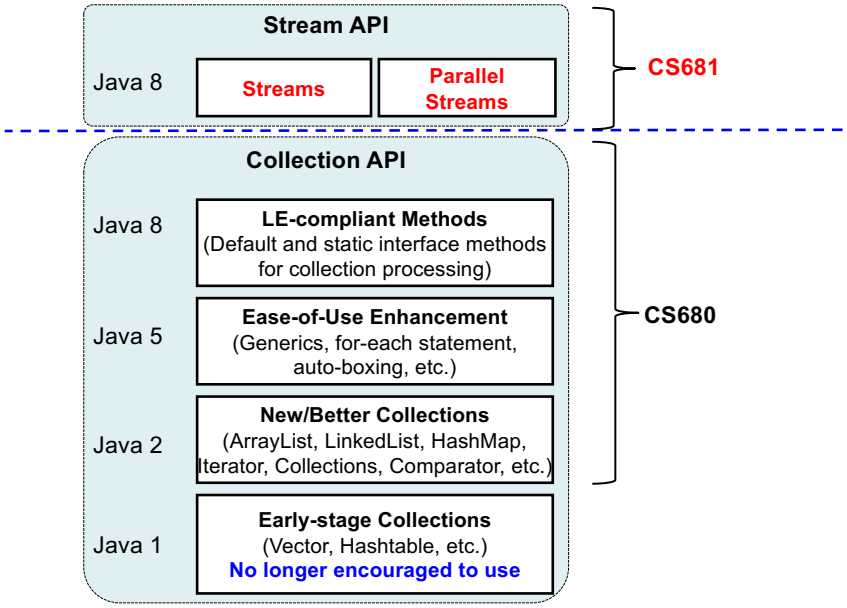
Course Topics: Advanced Software Engineering

- Functional programming with lambda expressions
- Concurrent object-oriented programming (multi-threading with Java)

Course Topic #1

- Functional programming with lambda expressions
  - Continuation from CS680
  - Collection processing with LEs
    - Stream API, which heavily uses LEs
    - Design patterns (data processing patterns); e.g., MapReduce

Collection and Stream APIs in Java



Please Note…

- You are expected to know some basics about LEs in Java.
  - i.e., topics covered in CS680
- CS681 will cover more advanced topics atop the basic understanding.
  - See CS680 lecture notes and refresh your memory about Java LEs, if necessary.

Course Topic #2

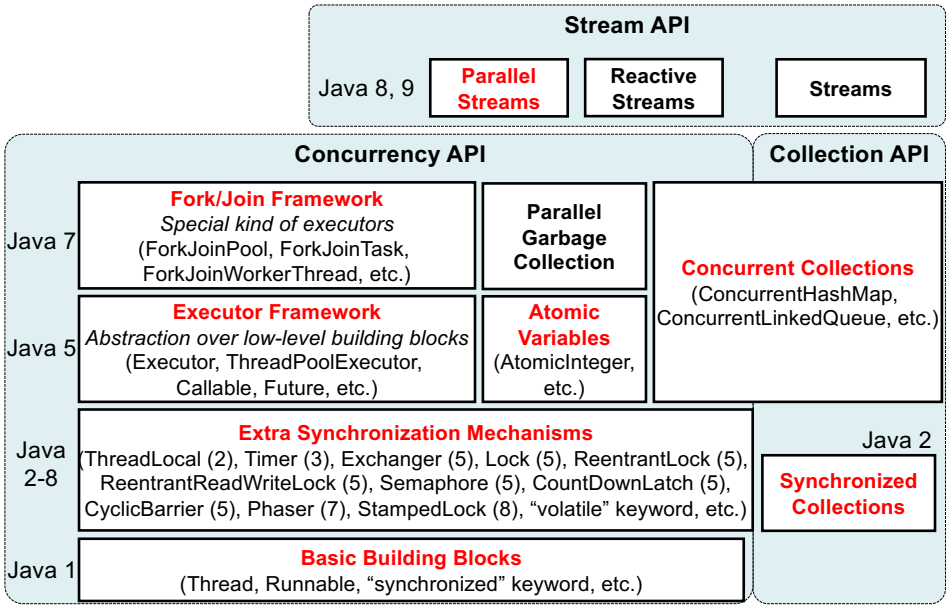
- Concurrent programming (multi-threading)
  - Mechanisms, data structures, APIs and frameworks for concurrency (multi-threading) in Java
  - Concurrent object-oriented design patterns
    - e.g., Concurrent *Singleton*, Concurrent *Observer*, Concurrent *Visitor*, Concurrent *MapReduce*, *Producer-Consumer*
  - Concurrency with LEs
    - With concurrency-aware collections, parallel streams, etc.

Concurrency as a Part of SE? Yes!

- Concurrency has a long history.
  - Concurrency in programming: Since mid 60s
    - PL/I ("TASK", '65), Concurrent Pascal ('75), Concurrent Smalltalk ('86), Objective-C ('86), Java ('96) , …, C# ('02), Erlang ('06), …, etc.
  - Concurrency in OSes: Since 70s
    - Multix ('70), Mach (cthreads, '85) , NeXTSTEP (cthreads, '86), OS/2 ('87), Solaris libthread ('93), Windows NT ('94), pthreads ('95), Windows ('95), Mac OS X ('01), Linux ('03), iOS, Android, …, etc.
- Concurrency was for specific groups of software engineers to develop specific types of software.
  - Early applications
    - High-performance and real-time computing in…
      - Defense, aviation, financial trading (algorithmic and high freq. trades), etc.

- Concurrency is now important and useful for most software engineers to develop a variety of software.
  - Not only "special" applications but also "normal" apps can enjoy, or even require, concurrency.
    - e.g., Web apps, smartphone/tablet apps, IoT
  - **Goals**: *Responsiveness* and *performance* improvement
    - Perform fine-grained multi-tasking
    - Do better I/O handling
    - Take advantage of multi-core CPUs

Concurrency API in Java



Course Work and Grading

- Course work
  - Lectures
  - Homework (coding in Java)
- Grading factors
  - Homework (50%)
  - Quizzes (50%)
- No midterm and final exams.

Homework

- There are 2 **FIRM** deadlines for HW submission
  - October 25 (Sun), midnight
    - Submit your solutions for the HWs given by mid October.
  - End of the semester
    - Specific date to be announced later.
    - Submit your solutions for the remaining HWs.
  - Firm deadlines are firm. (No extensions will be given.) Will NOT grade any late submissions.

- Place your HW solutions at a private GitHub repo and share it with me.
  - Share source code (.java files) only.
    - No binary code (e.g., .class and .jar files).
    - Will not grade any submissions that have binary code.
  - My GitHub account is jxsboston.
- Use Ant for every HW submission.
  - No need to use JUnit in CS681.
  - You can earn some extra points if you use JUnit.
- Questions:
  - Email: **Junichi.Suzuki@umb.edu**
  - Office hours: 4pm to 5:30pm, Tue and Thu

Suggestions

- Try your best to turn in your solutions early and regularly as HW assignments are given.
  - Rather than waiting for the deadline to turn them in.
- If you turn in your solutions early…
  - you can ask for my feedback about them, so you can revise them before the deadline.
- If you turn in your solutions regularly…
  - you can earn some extra points.

Important Notice

- You must work alone from scratch for each HW.
  - You can discuss HW assignments with others though.
  - NEVER start with anyone else's (or AI's) code. You must write your code yourself.
- It is an academic crime to
  - Copy (or steal) someone else's code and submit it as your own work.
  - Allow someone else to copy (or steal) your code and submit it as his/her work.
- You will end up with a serious situation if you commit this crime.
  - The University, College and Department have no mercy about it.

- Even if you believe you worked on a HW solution yourself, you are placed in a plagiarism case as far as it is somehow identical to someone else's solution.
  - NEVER start with anyone else's (or AI's) code.

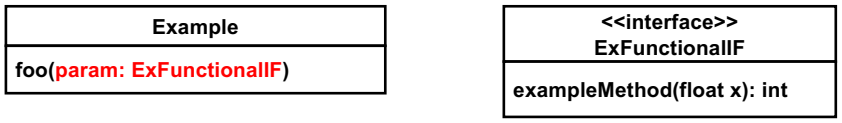
Recap: Lambda Expressions in Java

Benefits of Using Lambda Expressions

- Can make your code more concise (less repetitive)
  - *Callback* functions/methods as lambda expressions
    - e.g., Comparator (*Strategy*), *Command*
- Can enjoy the power of functional programming
  - e.g., higher-order functions (c.f. `Comparator.comparing()`)
- Can gain a new way to process collections
  - Newly-added *default methods* that accept lambda expressions
  - "Internal" iteration as opposed to traditional "external" iteration
  - Collection streams (Stream API)
    - Enables Map-Reduce data processing (a topic in CS681)
- Can simplify concurrent programming (multi-threading)
  - Repeatedly and concurrently executed code (block) as a lambda expression (a topic in CS681)

Where/When to Use Lambda Expressions

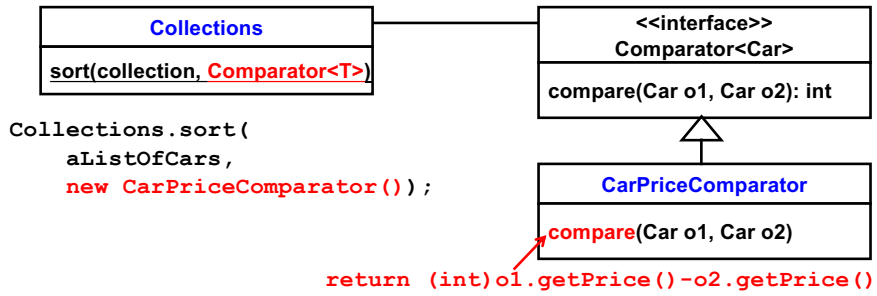
- A functional interface is given.
  - An interface that has an abstract method
- There is a method that expects a parameter that is typed with that functional interface.
- You can pass a LE to the method as a parameter value.
  - The LE must follow the abstract method's signature.




```
anExample.foo( (float x)-> {...} );
```


An Example with Comparator

- Traditional coding scheme:




```
Collections.sort(
    aListOfCars,
    new CarPriceComparator());
```

```
return (int)o1.getPrice()-o2.getPrice()
```

- Modern coding scheme with a LE:

```
Collections.sort((Car o1, Car o2)->{
                    return (int)o1.getPrice()-o2.getPrice();)
```

What does `Collections.sort()` do?

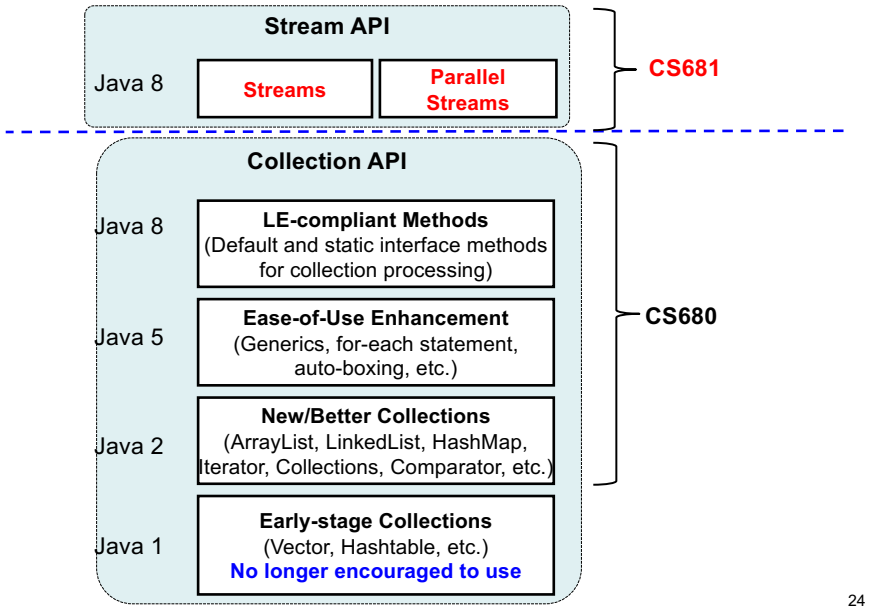
```
• class Collections
    static ... sort(List<T> list, Comparator<T> c){
        for each pair (o1 and o2) of elements in list{
            int result = c.compare(o1, o2);
            if(result < 0){
                ...
            }else if(result > 0){
                ...
            }else if(result==0){
                ...
      } } }
```


Stream API

New Way of Collection Processing with LEs

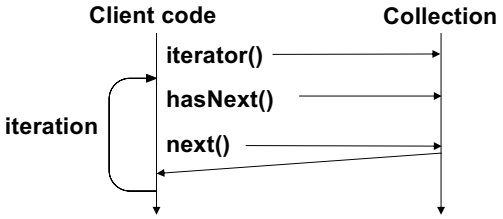
- Java 8 made major improvements to the Collection API by
  - Adding new static and default methods in existing interfaces
    - Those new methods allow you to pass LEs to your collection object.
    - e.g. `Iterable.forEach()`
    - c.f. CS680 lecture notes
  - Adding **streams**.
    - **`java.util.stream.Stream<T>`**
      - Has many methods that take care of common operations to be performed on collection elements (T).

Collection and Stream APIs in Java

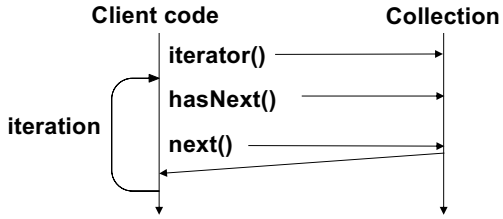


Traditional Way of Collection Processing

- ***External*** iteration: Iterate over a collection and perform an operation on each element in turn.
  -
```
Iterator<String> iterator = strList.iterator();
while( iterator.hasNext() ) {
    System.out.println(iterator.next()); }
```
- Iteration runs outside of a collection.
- Need to write boilerplate code whenever you need to iterate over the collection.



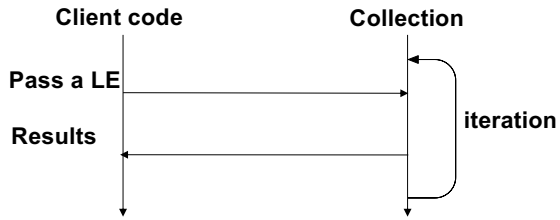
- ***External*** iteration: Iterate over a collection and perform an operation on each element in turn.
  -
```
int count = 0;
Iterator<Car> it = carList.iterator();
while( iterator.hasNext() ){
    Car car = iterator.next();
    if( car.getPrice() < 5000 ) count++; }
```
  - Iteration runs outside of a collection.
  - Need to write a lot of boilerplate code whenever you need to iterate over the collection.



- The loop mixes up *what you want to do on a collection* and *how you do it*.
  - Not that easy to understand and maintain because "what" is often obscured by "how." ("How" is often emphasized too much than "what.")
- Inherently serial
  - Hard to make it concurrent/parallel.

New Way of Collection Processing (1)

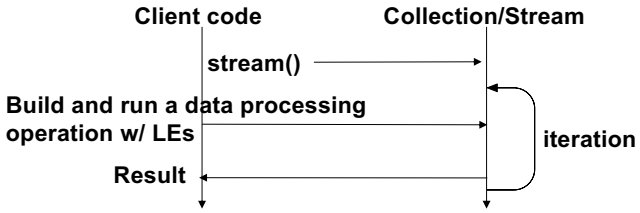
- ***Internal*** iteration:
  - `Iterable.forEach()`
    - Does not return an `Iterator` that externally controls the iteration
    - Creates an equivalent object, which works inside collection.
      - A collection internally uses the iterator-like object to perform iteration
  - `strList.forEach( (String i)->System.out.println(i) )`



New Way of Collection Processing (2)

- ***Internal*** iteration:
  - `stream()`: Plays a similar role to the call of `iterator()`
    - Does not return an `Iterator` that externally controls the iteration
    - Returns a `Stream`, which helps build a complex data processing operation on a collection and runs it in the collection.

```
long count = carList.stream()
                    .filter( (Car car)-> car.getPrice()<5000 )
                    .count();
```



- Client code simply states "what" you want to do on a collection. "How" is hidden.
  - Collection processing looks more declarative, not procedural.
    - c.f. SQL statements

```
long count = carList.stream()
                    .filter( (Car car)-> car.getPrice()<5000 )
                    .count();
```

- Get a stream that contains `Car` instances.
- Filter the stream to keep the `Car` instances that are less expensive than $5000
- Count the number of remaining `Car` instances in the steam.

Functional Interfaces

- Since its version 8, Java extensively uses ***functional interfaces*** in many APIs.
  - e.g., `Comparator<T>`: Used for a specific purpose:
    - Comparison of two data (objects)
    - c.f. `Collections.sort()`

- In addition to *special-purpose* functional interfaces, Java has *general-purpose* ones that can be generally used in many scenarios or for many purposes.

Important General-Purpose Functional Interfaces

Function<T,R>
Consumer<T>
Predicate<T>
Supplier<T>
UnaryOperator<T>
BinaryOperator<T>
BiFunction<U,T>

All available in `java.util.function`

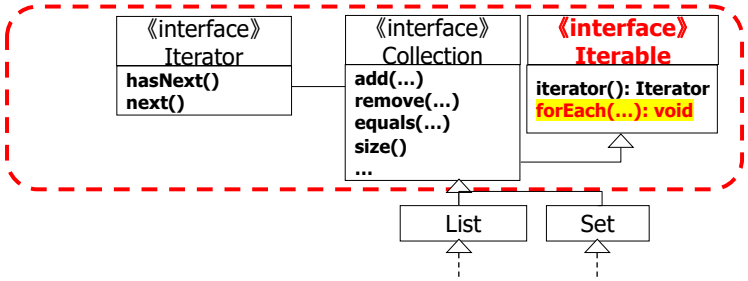
- Just like special-purpose functional interfaces…
- Each general-purpose functional interface helps you define and use a "function" as the form of a lambda expression.
  - Anonymous
  - Zero or more inputs
  - Zero or more outputs

- To understand what inputs/outputs the function is supposed to take/return, check out the (only) abstract method of that functional interface.
  - e.g., `Comparator<T>`
    - `int compare(T o1, T o2)`

	Params	Returns	Example use case
Function<T,R>	T	R	Get the price (R) from a Car object (T) Generate a function (R) from another (T) Comparator.comparing(), Map.computeIfAbsent(), Map.computeIfPresent()
Consumer<T>	T	void	Print out a collection element (T). Iterable.forEach()
Predicate<T>	T	boolean	Has this car (T) had an accident? Collection.removeIf()
Supplier<T>	NO	T	A factory method. Create a Car object and return it.
UnaryOperator<T>	T	T	Logical NOT (!) List.replaceAll()
BinaryOperator<T>	T, T	T	Multiplying two numbers (*)
BiFunction<U,T>	U, T	R	Return TRUE (R) if two params (U and T) match. Map.compute()

An Example General-Purpose Functional Interface: `Consumer`

- **`Iterable<T>`**
  - **`default void forEach(Consumer<T> action)`**
    - Applies a given function on each element of a collection that implements **`Iterable`**.




```
LinkedList<String> strList = ...
strList.forEach( (String s)->System.out.println(s) );
```


- **`Consumer<T>`**: Functional interface
  - Allows **`forEach()`** to receive a lambda expression.
  - Represents a function (LE) that accepts a parameter (T) and returns nothing.
    - The LE receives a collection element as a parameter and specifies an action to be applied to the collection element as its code block.

```
LinkedList<String> strList = ...
strList.forEach( (String s)->System.out.println(s) );
```

	Params	Returns	Example use case
Consumer<T>	T	void	Print out a collection element (T). Iterable.forEach()

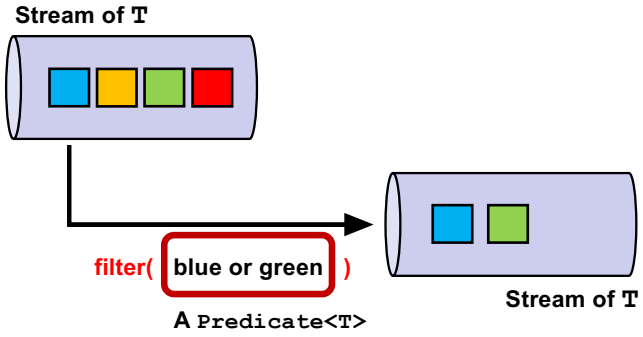
Streams and Collections

- Interface **`Collection<T>`**
  - **`default Stream<T> stream()`**
    - Returns a stream that uses this collection as its data source.

- **`java.util.stream.Stream<T>`**
  - **`Stream<T> filter(Predicate<T> predicate)`**
    - Returns a stream consisting of the elements of this stream that match a given predicate (i.e., filtering criterion).
  - **`long count()`**
    - Returns the count of elements in this stream.

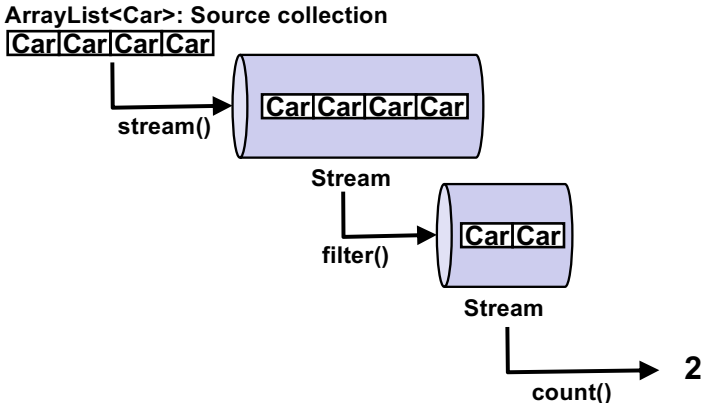
```
long count = carList.stream()
                    .filter( (Car car)-> car.getPrice()<5000 )
                    .count();
```

	Params	Returns	Example use case
Function<T,R>	T	R	Get the price (R) from a Car object (T) Generate a function (R) from another (T)
Consumer<T>	T	void	Print out a collection element (T)
Predicate<T>	T	boolean	Has this car (T) had an accident?
Supplier<T>	NO	T	A factory method. Create a Car object and return it.
UnaryOperator<T>	T	T	Logical NOT (!)
BinaryOperator<T>	T, T	T	Multiplying two numbers (*)



Stream Pipeline

- Multiple streams can be pipelined.
  - `long count = carList.stream()`
    `.filter( (Car car)-> car.getPrice()<5000 )`
    `.count();`
- Streams do NOT modify their source collection.



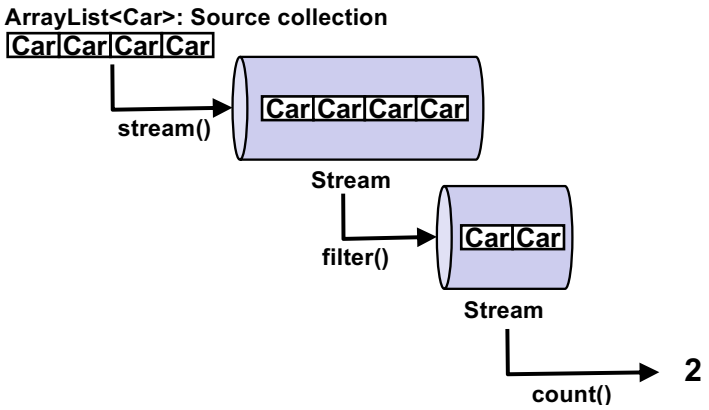
- Common steps to pipeline streams
  - Build a stream on a source collection
  - Perform zero or more *intermediate* operations
    - Each intermediate operation returns a **`Stream`**.
  - Perform a *terminal* operation
    - A terminal operation returns non- **`Stream`** value or void.

- `long count = carList.stream()`
  `.filter( (Car car)-> car.getPrice()<5000 )`
  `.count();`

How Many Traversals?

- Traditional
  - ```
    int count = 0;
    Iterator<Car> it = carList.iterator();
    while( iterator.hasNext() ){
        Car car = iterator.next();
        if( car.getPrice() < 5000 ) count++;  }
    ```
  - Traversing the list once .
- New
  - `long count = carList.stream()`
    `.filter( (Car car)-> car.getPrice()<5000 )`
    `.count();`
  - Traversing the list twice?
  - No, only once!
    ```

- This conceptual diagram is used to intuitively understand the structure and behavior of a stream pipeline.
  - Real traversal execution (internal iteration) is a bit different (more efficient)



Lazy and Eager Operations

- All intermediate operations are *lazy*.
- All terminal operations are *eager*.
- `filter()`: *intermediate* operation (lazy)
  - Does NOT perform filtering immediately when it is called.
  - Just prepares the filtering task and *delays* the task's execution until a terminal operation is invoked.
- `count()`: *terminal* operation (eager)
  - Is executed immediately when it is called.

```
long count = carList.stream()
                    .filter( (Car car)-> car.getPrice()<5000 )
                    .count();
```


- No intermediate operations are executed until a terminal operation is called.

```
... = carList.stream()
            .filter( (Car car)-> car.getPrice()<5000 );
```

  - The filtering operation is never executed.

```
... = carList.stream()
            .filter((Car car)->{
                    System.out.println(car.getPrice());
                    return car.getPrice()<5000} );
```

  - Nothing is printed out.
  - The filtering operation is never executed.

Exercise

- Write and run your own code based on the sample code in this lecture note.

Building a Stream

- Interface `Collection<T>`
  - `default Stream<T> stream()`
    - Returns a stream that uses this collection as its data source.

-
```
long count = carList.stream()
                    .filter( (Car car)-> car.getPrice()<5000 )
                    .count();
```


Important Methods to Build a Stream

- `of(T... values)`: a static method of `Stream`
  - Builds a stream with values (not a collection) as source data
    - `T`: Type of Stream elements (same as `T` in `Stream<T>`)
    - `T... values` is a syntactic sugar for `T[] values`.

Important Methods to Build a Stream

- `of(T... values)`: a static method of `Stream`
  - Builds a stream with values (not a collection) as source data
    - `T`: Type of Stream elements (same as `T` in `Stream<T>`)
    - `T... values` is a syntactic sugar for `T[] values`.

  -
```
List<String> collected =
                    Stream.of("u", "m", "b")
                          .map((String str)-> str.toUpperCase())
                          .collect( Collectors.toList() );
// a list of "U", "M" and "B" is returned.
```

  -
```
String[] strs = {"u", "m", "b"};
List<String> collected =
                    Stream.of(strs)
                          .map((String str)-> str.toUpperCase())
                          .collect( Collectors.toList() );
// a list of "U", "M" and "B" is returned.
```


- `concat(Stream<T> a, Stream<T> b)`: a static method of `Stream`
  - Concatenates two streams into a single (new) stream.

  -
```
LinkedList<Car> usedCars = ...
LinkedList<Car> newCars = ...
Stream<Car> cars =
                Stream.concat(usedCars.stream(), newCars.stream());
```

- **`generate(Supplier<T> s)`**: a static method of **`Stream`**
  - Returns an infinite stream in which each element is generated by using a given **`Supplier`** repeatedly.

	Params	Returns	Example use case
Supplier<T>	NO	T	A factory method. Create a Car object and return it.

- **`generate(Supplier<T> s)`**: a static method of **`Stream`**
  - Returns an infinite stream in which each element is generated by using a given **`Supplier`** repeatedly.

```
Stream<Double> randomNums = Stream.generate( ()-> Math.random() );
// infinite sequence of random numbers are generated.
```

```
Stream<Double> randomNums = Stream.generate( Math::random );
// infinite sequence of random numbers are generated.
```

```
Stream<Double> randomNums = Stream.generate( ()-> Math.random() )
                                  .limit(100);
// First 100 random numbers are selected and returned.
```

	Params	Returns	Example use case
Supplier<T>	NO	T	A factory method. Create a Car object and return it.

- *Method references* in lambda expressions
  - *object::method*
    - **`System.out::println`** (**`System.out`** contains an instance of **`PrintStream`**.)
    - **`(int x) -> System.out.println(x)`**
  - *Class::staticMethod*
    - **`Math::max`**
    - **`(double x, double y) -> Math.max(x, y)`**
  - *Class::method*
    - **`Car::getPrice`**
    - **`(Car car)-> car.getPrice()`**
    - **`Car::setPrice`**
    - **`(Car car, int price)-> car.setPrice(price)`**

- **`iterate(T seed, UnaryOperator<T> f)`**: a static method of **`Stream`**
  - Returns an infinite stream produced by applying a given **`UnaryOperator`** to the initial element **`seed`** iteratively.
    - Generated elements: seed, f(seed), f(f(seed)), …

	Params	Returns	Example use case
UnaryOperator<T>	T	T	Logical NOT (!)

- `iterate(T seed, UnaryOperator<T> f)`: a static method of `Stream`
  - Returns an infinite stream produced by applying a given `UnaryOperator` to the initial element `seed` iteratively.
    - Generated elements: seed, f(seed), f(f(seed)), …

```
Stream<Integer> integers =
                    Stream.iterate( 0, (Integer i)-> i+1 );
// Infinite sequence of 0, 1, 2, 3...

Stream<Integer> oddNums =
                    Stream.iterate( 1, (Integer i)-> i+2 );
// Odd numbers: 1, 3, 5, 7...

Stream<Integer> oddNums =
                    Stream.iterate( 1, (Integer i)-> i+2 );
                          .skip(2);
// First 2 odd numbers are removed: 5, 7...
```

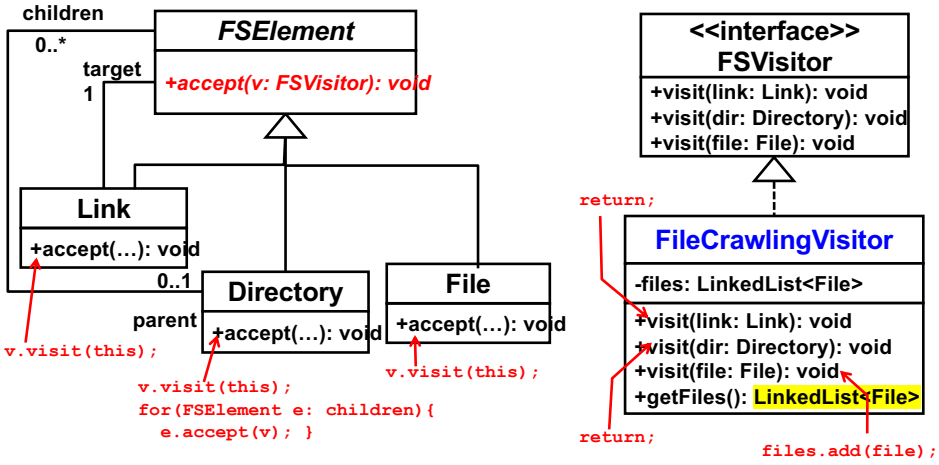
	Params	Returns	Example use case
UnaryOperator<T>	T	T	Logical NOT (!)

- `iterate(T seed, UnaryOperator<T> f)`: a static method of `Stream`
  - Returns an infinite stream produced by applying a given `UnaryOperator` to the initial element `seed` iteratively.
    - Generated elements: seed, f(seed), f(f(seed)), …


```
Stream<Integer> integers =
                    Stream.iterate(0, (i)-> i+1);
// Infinite sequence of 0, 1, 2, 3...

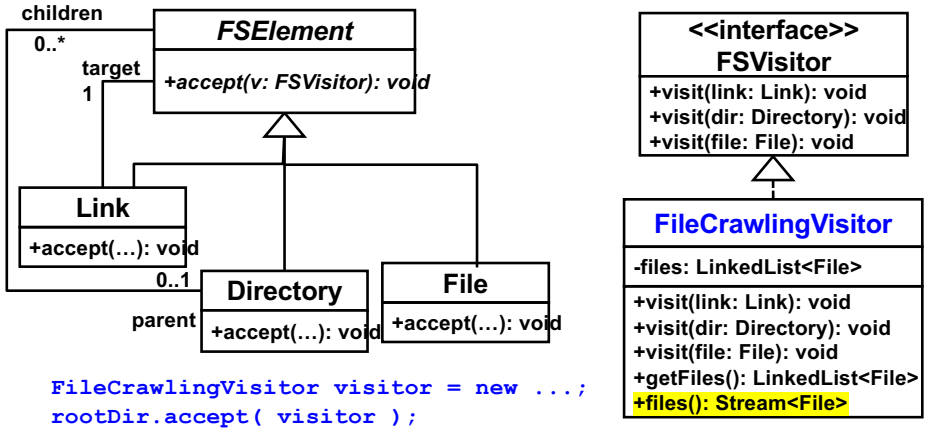

Integer seed = 0;
LinkedList<Integer> integers = new LinkedList<>();
integers.add(seed);
Integer i = seed;
for(;;){
  i = i+1;
  integers.add(i);
}
```


HW 1: Revise FileCrawlingVisitor




```
FileCrawlingVisitor visitor = new FileCrawlingVisitor();
rootDir.accept( visitor );
visitor.getFiles();
```


- Add a new method to `FileCrawlingVisitor`.
  - `files()`, which returns a stream of `Files`.
- Keep all existing code (CS680 code) as they are.




```
FileCrawlingVisitor visitor = new ...;
rootDir.accept( visitor );
visitor.files()
       ...
       .count();
```

- Find the number of files that
  - were created after a certain `LocalDateTime`.
  - have a certain file extension (e.g. .java).

- Use the file system structure (i.e. test fixture) that you implemented in CS680.
  - Assign values to data fields (size, etc) as you like.

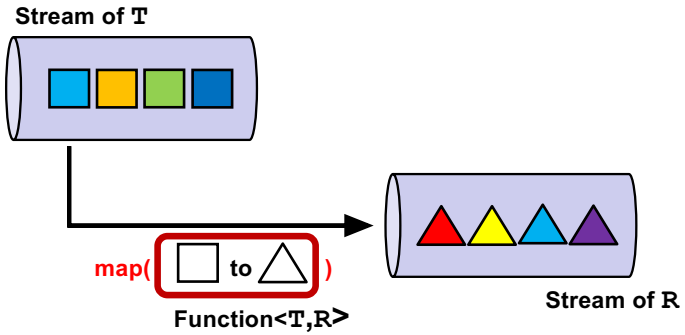
Other Important Methods in Stream

- `map(Function<T,R>)` : *intermediate* operation
  - Performs a stream-to-stream transformation
    - Takes a `Function` that converts a value of `T` to another of `R`.
      - `T` and `R` can be different types.
    - Applies the function on stream elements one by one.
    - Returns another stream of new values.
      - The # of elements does NOT change in b/w the input and output streams.

```
List<String> collected =
                    Stream.of("u", "m", "b")
                          .map((String str)-> str.toUpperCase())
                          .collect( Collectors.toList() );
  // a list of "U", "M" and "B" is returned.
```

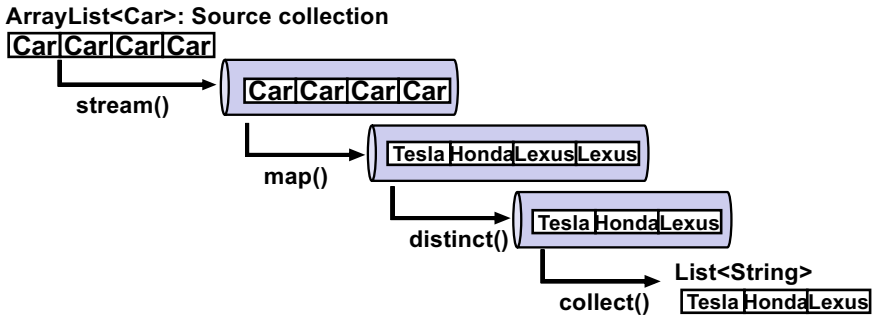
	Params	Returns	Example use case
Function<T,R>	T	R	Get the price (R) from a Car object (T)

	Params	Returns	Example use case
Function<T,R>	T	R	Get the price (R) from a Car object (T) Generate a function (R) from another (T)



- `distinct()`: *intermediate* operation
  - Removes redundant elements and returns a stream consisting of distinct elements

```
List<String> makes = cars.stream()
                         .map( (Car car)-> car.getMake() )
                         .distinct()
                         .collect(Collectors.toList());
```



- `sorted(Comparator)`: *intermediate* operation
  - Sorts stream elements according to a given `Comparator` and returns the sorted stream.

```
- List<Float> prices =
    cars.stream()
        .map( (Car car)-> car.getPrice() )
        .distinct()
        .sorted( (Double price1, Double price2)-> price1-price2 )
        .collect( Collectors.toList() );
```


- `sorted(Comparator)`: *intermediate* operation
  - Sorts stream elements according to a given `Comparator` and returns the sorted stream.

```
- List<Car> sortedCars =
    cars.stream()
        .sorted( (Car car1, Car car2)->
                                    car1.getPrice()-car2.getPrice()) )
        .collect( Collectors.toList() );
```

```
- List<Car> sortedCars =
    cars.stream()
        .sorted( Comparator.comparing((Car car)-> car.getPrice()) )
        .collect( Collectors.toList() );
```

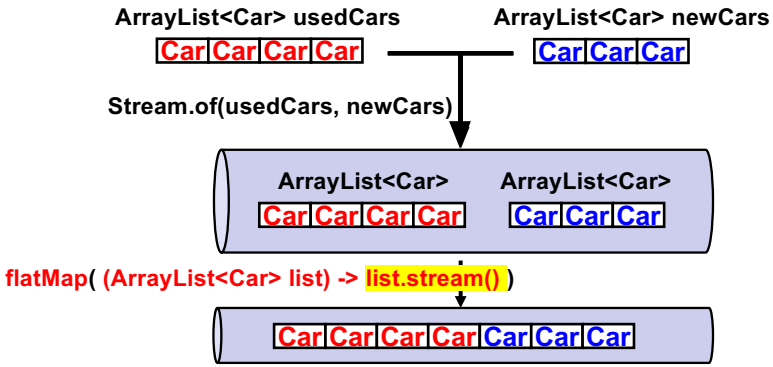
  - `comparing()`: higher-order function; c.f. CS680

HW 2

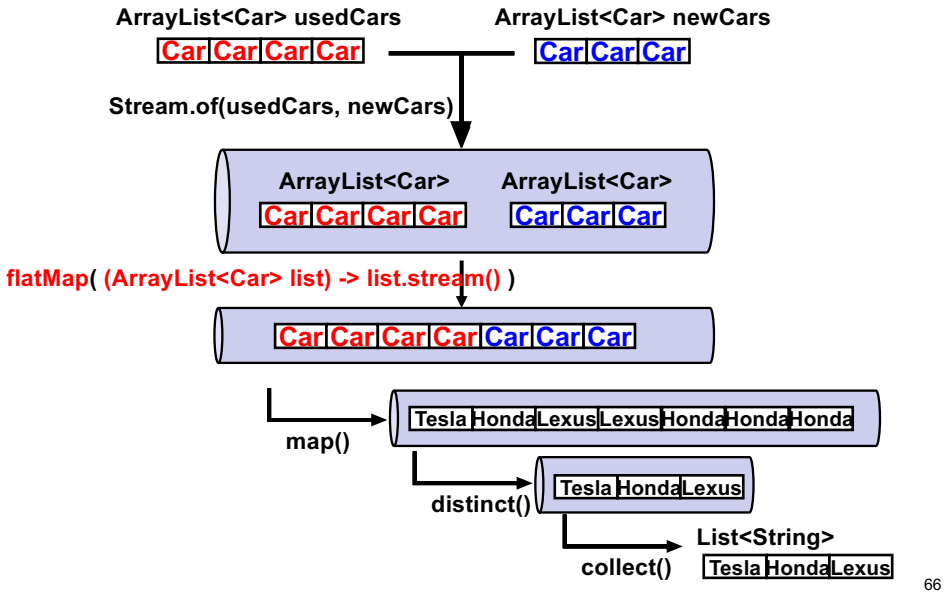
- Recall a HW in CS680: Sorting `Car`s with `Collections.sort()` and 4 LEs.
  - 4 LEs implemented 4 different ordering policies: price-, year-, mileage-, and domination count-based sorting.
- Do the same (i.e., sort `Car`s) with Stream API.
  - c.f. `Stream.sorted()`
  - Implement 4 different sorting policies

- `flatMap(Function<T,R>)`: *intermediate* operation
  - Accepts a `Function` that converts each element of a steam to a separate steam
  - Concatenates all the converted streams into a single stream.
  - `R` must be a stream.

```
- ArrayList<Car> usedCars = ...
  ArrayList<Car> newCars = ...
  List<String> makes =
              Stream.of(usedCars, newCars)
                    .flatmap( (ArrayList<Car> list)-> list.stream() )
                    .map( (Car car)-> car.getMake() )
                    .distinct()
                    .collect(Collectors.toList());
```



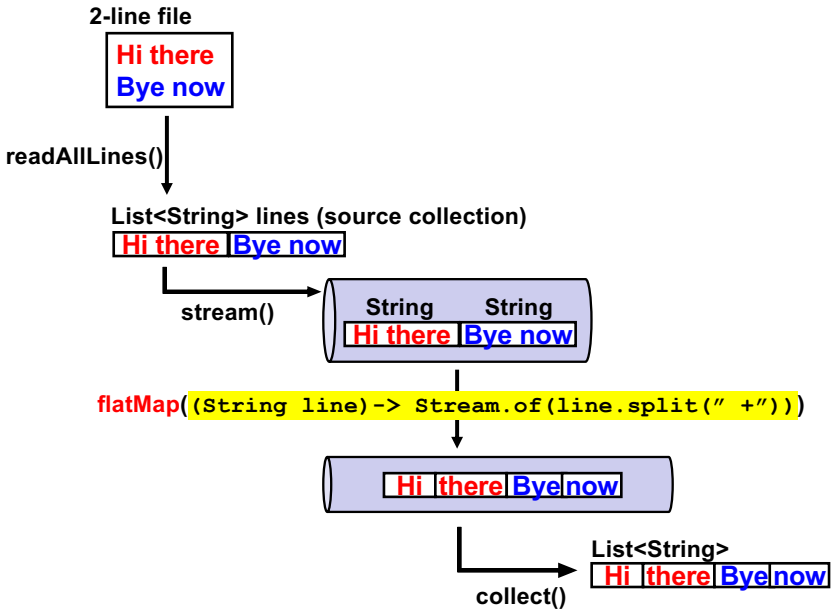

```
ArrayList<Car> usedCars = ...
ArrayList<Car> newCars = ...
List<String> makes =
            Stream.of(usedCars, newCars)
                  .flatmap( (ArrayList<Car> list)-> list.stream() )
```


```
Path path = Paths.get("/Users/jxs/temp/test.txt");
List<String> lines = Files.readAllLines(path);
List<String> words =
      lines.stream()
           .flatMap( (String line)-> Stream.of(line.split(" +")) );
           .collect(Collectors.toList());
```

- New I/O (NIO) : `java.nio`
  - Includes `Path`, `Paths`, `Files`, etc.
  - c.f. CS680 slides

- `String.split()`
  - takes a regular expression and returns an array of Strings.
  - `" +"` means splitting a string (`line`) with a space as a delimiter.
    - e.g., "Hi there" → ["Hi", "there"]



- `max(Comparator<T>)`: *terminal* operation
  - Returns the maximum value according to a given `Comparator`.
- `min(Comparator<T>)`: *terminal* operation
  - Returns the minimum value according to a given `Comparator`.

```
Car searchResult =
    cars.stream()
        .filter( (Car car)-> !car.hadAccidents() )
        .min( Comparator.comparing( (Car car)->car.getPrice() ) )
        .get();
```

- `max()` and `min()` return an `Optional<T>`.
  - An `Optional` represents a value that may or may not exist.
    - It does not exist (`==null`) if `max()` or `min()` is called on an empty steam.

- `get()` of `Optional<T>`
  - If this `Optional` contains a value, returns it.
  - Otherwise, throws `NoSuchElementException`.
- `isPresent()` of `Optional<T>`
  - Checks if this `Optional` contains a value.

```
Optional<Integer> p = cars.stream()
                          .min( ... );
if(p.isPresent()){ System.out.println( p.get() ); }
```

- `ifPresent(Consumer)` of `Optional<T>`
  - If this `Optional` contains a value, apply an LE on it.
  - Otherwise, does nothing.

```
cars.stream()...ifPresent(val -> System.out.println(val));
```

- `orElse(T)` of `Optional<T>`
  - If this `Optional` contains a value, returns it.
  - Otherwise, returns the supplied value.

```
Integer min = cars.stream()
                  .min( ... )
                  .orElse(0);
```


- `forEach(Consumer<T>)`: *terminal* operation
  - Applies an LE on each stream element.

```
cars.stream()
    .map( (Car car)-> car.getMake() )
    .distinct()
    .forEach( (String autoMaker)-> System.out.println(autoMaker));
```

  - `forEach()` does NOT retain the order of elements,
    - even though the source collection (`cars`) is ordered.
- All other methods of `Stream` retains the order of elements, if the source collection is ordered.

API Methods that Return Streams

- Many API methods return streams.
- `java.nio.file.Files`
  - A utility class to process files and directories.
    - Java NIO: c.f. CS680
  - `lines(Path path)`: Reads all lines from a file as a Stream.

```
Path path = Paths.get("/Users/jxs/temp/log.txt");
try( Stream<String> lines = Files.lines(path) ){
  long postsCount =
    lines.filter( (String line)-> line.contains("POST") )
         .count(); }
```

    - Try-with-resources statement: c.f. CS680

```
try( Stream<String> lines = Files.lines(path) ){
  lines.skip(100)
       .limit(100)
       .forEach(System.out::println); }
```

- **java.io.BufferedReader**
  - Performs buffering to read characters from a file.

```
try( BufferedReader reader =
        Files.newBufferedReader( Paths.get("test.txt")) ){
   Boolean commented =
       reader.lines()
             .anyMatch( line-> line.startWith("#") ); }
```


Just in case: Buffering

- At the lowest level, read/write operations deal with data *byte by byte*, or *char by char*.
- Inefficient if you read/write a lot of data.
- Buffering allows read/write operations to deal with data in a coarse-grained manner.
  - Chunk by chunk, not byte by byte or char by char
  - Chunk = a set of bytes or a set of chars
    - The size of a chunk: 512 bytes by default, but configurable

Just In Case…

- **Stream<T> sorted(Comparator<? super T> comparator)**
  - "? super T" means any super type (super class) of T.
- **Stream<R> map(Function<? Super T, ? extends R> mapper)**
  - "? super T" means any super type (super class) of T.
  - "? extends R" means any sub type (subclass) of R.

```
List<Car> cars = ...;
String result = cars.stream()
                    .map( (Object car)-> car.toString() )
                    ...
```


Major General-Purpose Functional Interfaces

	Params	Returns	Example use case
Function<T,R>	T	R	Get the price (R) from a Car object (T) Generate a function (R) from another (T) Comparator.comparing(), Map.computeIfAbsent(), Map.computeIfPresent()
Consumer<T>	T	void	Print out a collection element (T). Iterable.forEach()
Predicate<T>	T	boolean	Has this car (T) had an accident? Collection.removeIf()
Supplier<T>	NO	T	A factory method. Create a Car object and return it.
UnaryOperator<T>	T	T	Logical NOT (!) List.replaceAll()
BinaryOperator<T>	T, T	T	Multiplying two numbers (*)
BiFunction<U,T>	U, T	R	Return TRUE (R) if two params (U and T) match. Map.compute()